

PROTECT-DB: Protecting Data using Replicated State Machines: Efficient Corruption Detection & Recovery

Anant Utgikar
anant1@iitb.ac.in
CSE Department, IIT Bombay
India

S. Sudarshan
sudarsha@cse.iitb.ac.in
CSE Department, IIT Bombay
India

Abstract

Data is critical for the operation of any organization and needs to be protected, especially against attacks that compromise the state of the database. In this paper, we explore an approach based on Byzantine-fault tolerant replicated state machines, built on top of a deterministic extension of PostgreSQL. Each replica deterministically executes transactions recorded in a shared log/blockchain. Our focus is on creating a practical system that is designed for efficient and quick detection of corruption, as well as quick repair concurrent with execution of transactions. We also present a performance study showing the efficiency and practicality of our approach. We believe our work lays the foundations for the practical use of the BFT replicated state machine approach in the context of databases.

PVLDB Reference Format:

Anant Utgikar and S. Sudarshan. PROTECT-DB: Protecting Data using Replicated State Machines: Efficient Corruption Detection & Recovery. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

1 Introduction

Data is critical for any organization, and there have been numerous incidents in recent years where attackers have broken-in and damaged data. Most databases support read/write access control. However, such access control does not protect against users who have administrator privileges on the database; in fact, most applications connect to databases via logins that can update any of the application data. A malicious user with administrative privileges could compromise the database contents. This could happen due to an intruder getting administrator credentials, or due to an insider attack. In this paper, we address the problem of detection and repair of such corruption of data.

One approach to handling the problem is to use a set of database replicas, with the security of each node configured independently with its own administration. It is harder for the attacker to break into the security of multiple nodes, thus providing protection from corruption as long as only a minority of nodes is compromised. If the replicas are under the same administration, a compromised user account will result in the compromise of all replicas. It may be noted that replication is used to provide redundancy in the event

of hardware failure, which can also be used for protection from attacks.

However, in typical implementations of database replication, updates happen at a single node and are then replicated to the other nodes. If the source of the update is compromised, data in all the replicas will also be compromised. Executing the updates independently on the database replicas is not an option since the transaction ordering may differ on different nodes, and further, some transactions may commit on one node but not on another.

The approach we follow is based on Byzantine-fault tolerant replicated state machines, where transactions are recorded in a shared log using consensus mechanisms, and are then executed on all replicas.

There are three major challenges in designing such a system. First, all nodes need to agree on the order of transactions submitted to the system. Second, all nodes must be kept synchronized while executing the same sequence of transactions. Third, we need to efficiently identify and recover from corrupted data/compromised nodes.

The first challenge, of ordering input requests, is addressed by using a shared log, through distributed consensus protocols such as Raft (which we use in our implementation). Alternatively, Byzantine consensus protocols, or permissioned block chains (with each block containing only 1 transaction, to minimize latency) can be used instead. The log may alternatively be created by an external system, for example, a matching engine in a stock market, with the transactions being recorded persistently at a share depository. All database replicas fetch transactions from this shared log and execute them independently.

The second challenge, of keeping databases synchronized, arises from the concurrent execution of transactions, which can result in different serialization orders in different replicas. Serial execution avoids this problem, but is too inefficient to be used in practice. Deterministic database models [1], [15], [14] allow some amount of concurrent execution for efficiency, but guarantee that the serialization order will be the same, and hence the result as well as the final state will be the same across all replicas.

The third and final challenge, which is the focus of this paper, is how to identify compromised nodes/corrupted data, and recover from corruption efficiently, even on very large databases.

In this paper, we describe an architecture for a system designed to deal with Byzantine data corruption, with a focus on simple, efficient, and practical schemes for corruption detection and recovery.

Our specific contributions are as follows.

- (1) We present (in Section 3) our system model, which provides a practical architecture for building database applications, by using a BFT replicated state machine approach on top

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

of existing databases. The model, also used in some other systems such as [18], is based on transaction code being executed at servers, while clients submit transaction requests that are basically API calls with parameters, digitally signed by the client.

We also present (in Section 4) a deterministic execution algorithm, which is based on Aria [15].

- (2) We present our corruption detection and recovery techniques in Section 5.

- (a) We describe two threat models (Section 5.1), one where database data is corrupted but system code is safe, and another where system code also could be compromised.

- (b) We discuss (in Section 5.2) two schemes to detect corruption of data. The first scheme (the passive scheme) checks the results of every query to detect any divergence, which would happen if some nodes get compromised and the data in the nodes get corrupted. If the majority of nodes are up and agree on the result of a transaction, that is accepted as the outcome. A node is identified as compromised if it gives a different result than the majority.

The second scheme (the active scheme) checks for divergence of database state across replicas, even if no query accesses the parts that have diverged. We compare database states on all nodes using a Merkle tree on each relation. To reduce Merkle root contention, we partition the records across multiple Merkle trees per relation.

- (c) We propose (in Section 5.3) an efficient and practical scheme for recovering from corruption, which can execute even as the system continues to process other transactions. This scheme depends on getting database snapshots, a feature that is supported by many databases such as PostgreSQL.
- (3) Replicated state machines depend critically on transactions being deterministic; yet bugs can cause non-deterministic behavior, which in turn can cause database state divergence. We propose (Section 6) a simple scheme for quick detection and recovery from execution of non-deterministic transaction, which can cause serious problem if it occurs and is not detected and fixed immediately.
- (4) We have implemented all our techniques. Section 7 describes details, including support for clients to submit transactions and get results back from the replicas, and support for recovery.

We use the Ratis implementation of Raft for crash-fault tolerant consensus, and Kafka for fault-tolerant asynchronous messaging. We also made small extensions to PostgreSQL to support deterministic execution based on the rw-conflict detection mechanism of Aria [15], implemented in PostgreSQL by AriaBC (github.com/zllai/AriaBC), and to implement Merkle trees.

- (5) We present a performance study in Section 8 that demonstrates that our implementation is very efficient, and can support a throughput of over 2500 transactions per second from the YCSB benchmark even on low-end hardware. Our

study shows that the overheads of deterministic execution and Merkle trees are quite low compared to the baseline performance of PostgreSQL. Raft and Kafka do impose overheads, and optimizing their performance to further improve throughput is an area of future work.

Our approach provides a practical and effective way to detect and recover from unauthorised updates to the database. We believe it can have significant real-world impact. Preventing unauthorised leakage of information is a separate problem and not the focus of this paper.

2 Related Work

We discuss related work on Byzantine-fault tolerant replicated state machines in Section 2.1, related work on deterministic execution, in Section 2.2. and other related work in Section 2.3.

2.1 Byzantine Fault Tolerance

Byzantine-fault tolerant replicated state machines are a well-known approach for keeping data safe from attacks. See [8] for a survey of the area from a systems perspective.

In a replicated state machine, a common input log of transactions is read by each of multiple replicas, and processed deterministically in the input log order. Depending on the application needs, the log could be generated by a crash tolerant consensus protocol like Raft, by a Byzantine-fault tolerant consensus protocol, such as those used in (permissioned) blockchains, or by any other application dependent mechanism.

Our work focuses on protecting the database state, given a sequential input log of transactions.

A Byzantine-fault tolerant replicated state machine guarantees data integrity, even if a few replicas are corrupted by an attacker, as long as less than a small minority (usually less than 1/3rd) are corrupted, See [8] for a survey of work in this area.

The bulk of the work on BFT replication has focused on non-database applications. Most BFT replication solutions process the log serially in each replica. While serial processing is acceptable in some situations, for general database applications, serial execution has a very negative impact on throughput.

A number of papers have addressed Byzantine fault tolerance for data storage systems, but most of them do not support SQL, and none of them address efficient corruption detection and recovery, which is very important for large databases. Basil [26] supports Byzantine fault tolerance, but on a key-value store that supports limited operations, Callinicos [20] also supports key-value stores with limited operations (“mini transactions”) and does not support SQL, while Janus [34] addresses asynchronous BFT. Augustus [21] presents a scalable storage system, supporting partitioning and replication, which is robust in the presence of Byzantine failures, but has restrictions on transactions. None of the above systems support SQL, nor do they discuss corruption detection and recovery, which is the focus of our paper.

Early work in BFT replication for databases includes [12], and [13], but these too assumed serial execution. HRDB [29] allows concurrent execution on SQL databases. It uses a primary database to execute transactions concurrently, and define a transaction ordering and forces replicas to follow the same order. HRDB depends on strict

2PL, whereas current generation databases often use (serializable) snapshot isolation. Subsequent work on BFT database replication includes Byzantium [22] and Mitra [16]. Byzantium executes transactions in three parts, begin, execution and commit. Begin and commit operations execute serially in the log order. Transaction execution can run concurrently, on a snapshot determined by the transaction start. Mitra [16] allows a coordinator to run transactions concurrently first, and then replicas follow the same order. Both these approaches support SQL queries. Pesto [27] extends the ideas behind Basil to support SQL, but requires significant changes to the database. However, none of the above mentioned approaches addresses corruption detection or recovery.

HyperLedger Fabric [3] uses an execute-order paradigm, where transactions are “simulated” on multiple servers (endorsers), and if the endorsers agree on the read and write sets, the transaction gets committed. The most closely related work to ours is [18], which builds a “blockchain relational database” on top of PostgreSQL. Our transaction model is similar to theirs. They support order-then-execute and execute-then-order models; we follow the first model. But neither [3] nor [18] considers corruption detection and recovery.

chainifyDB [24] addresses BFT on top of existing SQL databases. As also mentioned in [24], earlier work in this space does not address recovery, whereas chainifyDB does pay attention to recovery. However, recovery is based on database snapshots, which are very impractical for large databases both for creation and for copying over a network. (Their experiments were on 1,000,000 records which is relatively small.) In contrast we use Merkle trees for efficient recovery. Furthermore, they use triggers for tracking changes; an intruder who can modify the database can equally well disable triggers when modifying the database. In contrast, we use index update mechanisms to maintain Merkle trees, avoiding this vulnerability.

2.2 Deterministic Databases

The area of deterministic databases focuses on execution of transactions in a way that the serialization order respects the given input order, while supporting concurrent execution to improve throughput.

Early work in the deterministic database area included Calvin [28], and [23]. An overview of work on deterministic databases is provided in [2]. These papers address deterministic execution on a single node as well as deterministic execution in a partitioned-parallel database system.

Aria [15] is a deterministic execution protocol, where multiple transactions are batched and executed concurrently, on the same database snapshot. If a conflict between read-write sets is detected within a batch, the lower ID transaction is allowed to commit. Conflicting higher ID transactions are run again in the next batch, on a newer snapshot. The order is guaranteed to be the same on all replicas, although it may not match the given input order.

Harmony [14] is another deterministic system that also executes multiple transactions in a batch concurrently for better performance [32]. Harmony also reorders conflicting transactions by re-executing them in the next batch. The conflict detection mechanisms in Aria and Harmony handle predicate reads also. The authors

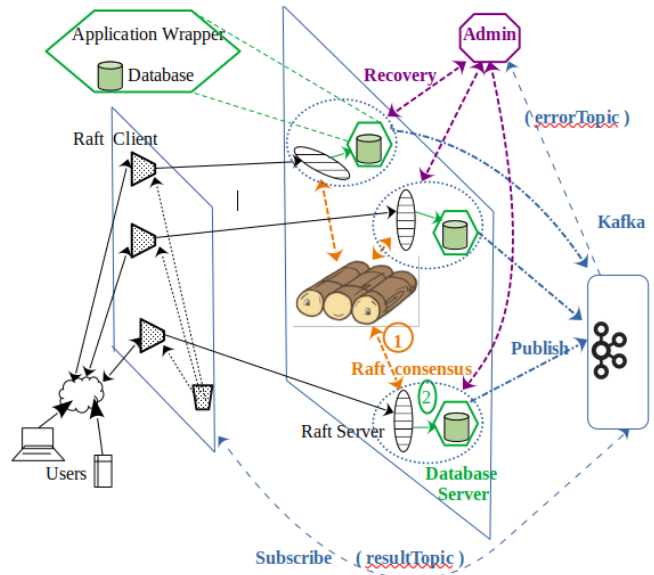


Figure 1: System model

of Harmony have developed an implementation of conflict detection as done in Aria, by localized modifications to the PostgreSQL code.¹ We use this conflict detection code in our implementation.

2.3 Other Related Work

Ledger databases such as [30] provide verification and auditing support, preserving historical information for forward integrity checking [5]. [30] does not support SQL, whereas SQL Ledger [4] provides forward integrity for updates on an SQL database using a blockchain. However, these systems require an audit process for detecting corruption, and meanwhile wrong answers may be returned. Further they do not provide any efficient means for recovery from corruption.

Protection of data in outsourced databases has received much attention in the past. Here, the model is that there is only one copy of the database, with data integrity verified by means of digital signatures, and query result correctness checked by sending integrity proofs or “verification objects” with the query results, which are then verified by the client. While there has been a lot of research in this area, e.g. [6], [17], among many others, all the existing results in this area work only for very limited query types, and are also very expensive, making them impractical, unlike BFT replicated state machine approaches such as ours.

TAPIR [33] provides transactional guarantees on top of inconsistent replication. However, it does not deal with Byzantine failure.

3 System Model

Figure 1 shows our system model pictorially. In our model, clients submit transactions using digitally signed messages, which are added to a shared log, created using a consensus mechanism or other mechanisms. In our implementation we use Ratis, an open source implementation of Raft.

¹Available at <https://github.com/zllai/AriaBC>.

Request	Client-ID	Sequence-Number	Transaction Statements	RSA-Signature
---------	-----------	-----------------	------------------------	---------------

Figure 2: Request format

Each replica in our Byzantine fault tolerance state machine replication model is implemented as an application server that provides a wrapper around a database system. The wrapper layer fetches each transaction from the shared log and processes the transaction. It performs authentication (using the signature), authorization checks, and then executes the transaction. Multiple threads are used to allow parallel execution of transactions, with the deterministic database component responsible for ensuring ordering. The results of transaction execution at each replica are sent back asynchronously to the client using Kafka’s pub-sub mechanism, to ensure fault tolerance.

We describe the components of the model in this section.

3.1 User Interaction Model

Users submit transactions through a client, which could be either their own device or through a web application. We assume for simplicity that clients are trusted and not compromised. Even if they are compromised, the damage is limited to operations within the authorization scope of the specific user.

As shown in Figure 2, each transaction contains (i) the user id, (ii) a sequence number that is unique for that user (to avoid duplication/replay attacks), (iii) the users transaction, which identifies the operation to be executed at the application server, along with its parameters, and (iv) a digital signature of the message contents.

Each Raft client also subscribes to Kafka to learn about the result of execution of the transaction at each replica. The client compares results from all replicas; as long as a majority of the full set of servers have sent the same value, it picks that value. Under the assumption that a majority of replicas are alive, accessible, and not corrupted, there would always be such a majority value, ensuring the system makes progress and does not block.

We discuss in Section 5.2 how to handle and resolve discrepancies if the results from different replicas differ. The comparison of results at the client can be optimized, by just sending a hash from all replicas except for one replica that sends the full result, and comparing the hash values and validating the result against the hash.

3.2 Transaction Log

Multiple users can simultaneously submit requests, with each containing one or more transactions. To ensure all replicas handle user requests in the same order, it is necessary that all replicas agree on the order of incoming requests. Thus, requests have to be added to a sequential log, which should be immutable for security reasons.

The input log can be generated by a Byzantine-fault tolerant consensus mechanism such as those used in (permissioned) blockchains, for e.g. [31]. If attacks on the consensus are not an issue of concern, and the focus is only on database state corruption, crash fault-tolerant consensus mechanisms like Raft [19] can be used, even though they are not Byzantine-fault tolerant.

There are also applications where the log is generated by an existing centralized system, but the state must be protected from attacks. For example, a stock market matching engine generates a sequential log of trades, which must then be processed by a depository which tracks who owns which shares; protecting the depository state from attack is a critical sub-problem. Where multiple parties are involved in a transaction, verification of consistency of such logs is done by periodically comparing the logs generated at the different systems, to detect and fix any inconsistencies, a process referred to as “reconciliation”.

In our implementation, we create the shared log by using the open-source Ratis implementation of the RAFT consensus protocol. Clients submit transactions to a Ratis client, which then sends the transactions to a Raft server. The Raft servers run the consensus protocol to create the shared log. Once a transaction is successfully committed in the log, it is executed on the replicas.

If we used blockchain-based mechanisms, the use of larger blocks can cause significant delays in transaction execution. With permissioned block chains, it is possible to use blocks containing only one or a few transactions, to avoid latency.

3.3 Transaction Processing at Replicas

When the Raft server detects that a transaction is committed in the log, it executes the transaction by sending it to the application server. We have an application server running on the same node as each Raft server; thus, each server sends the transaction to its local application server.

The application server first validates the digital signature, and performs authorization checks to determine if the user is authorized to execute the transaction contained in the request. If the checks are passed the transaction is executed in the application server, on the underlying database. Otherwise, the transaction is deemed to have failed.

Transaction execution in our model has to be deterministic, with the serialization order identical to the order in the log. We use deterministic database execution techniques to allow concurrent execution while ensuring deterministic results, as described later.

Once the transaction is executed, the application server sends the results to the client, but indirectly via a Kafka pub-sub system to handle any crashes. The results are signed by the server, and the client can authenticate the signature.

Each of the replicas will receive transactions in the same input order. Thus, after execution, the database state of all replicas will be identical, provided the initial state was the same, and the majority of replicas are not compromised. We will see in a later section how the corrupted replicas are handled and restored.

It is possible to use the database itself as an application server, but then transactions would be restricted to stored procedures supported by the database.

3.4 Detecting Data Corruption

Participating server replicas that are corrupted or compromised may deliver different query results than the healthy majority, which will provide identical results.

Clients compare the results received from all the replicas. As long as a majority of the complete set of replicas have sent the

same value, it is accepted, under the assumption that a majority of replicas are up and not corrupted. As long as only a minority of replicas are down or corrupted, the system will be able to make progress, else it may block. We discuss in Section 5.2 how to resolve discrepancies if the result hashes from different replicas differ.

As an optimization, only one of the replicas, chosen as the leader, can send the full result to the client, and others can simply send the result hash. The client can then compare the hashes with the locally verified hash of the full result from the leader. In case of any discrepancy, as long as a majority of the replicas sent the same hash value, the client can request one or more of the majority replicas to send the full result.

Update transactions are processed similarly to queries. The result of update SQL queries is usually very small, indicating the status of the operation and the number of rows updated.

In general, a user transaction executed in the application wrapper layer can be a combination of query and update operation. The application wrapper has the final choice in what result to send back to the client.

Our focus is on server compromise leading to data corruption at the server. However, it is also possible for a client to have been compromised. We note that if the client’s secret key has been leaked, any transaction sent ostensibly by that client would get validated. However, since these clients are typically end users who are authorized to only update their own data, the impact of such compromise would be limited to that client, thus minimizing risk to the overall system.

3.5 Key Management

Each client digitally signs the transaction with its private key. Similarly, each replica server digitally signs the execution result with its private key. Public keys of both sides must be stored in the recipients’ database or an external safe store. Any change in the public key should be done either with a certificate or through a trusted symmetric channel.

User keys being stored in the database, any update to keys is also a transaction on the blockchain. Thus, if a request signed with the old key arrives after the key-update request in the blockchain, it will be rejected.

4 Deterministic Execution Algorithm

Several schemes have been developed for deterministic execution, which allow concurrent execution; see e.g. [1]. These schemes typically allow transactions to execute concurrently, but commit serially, with transaction T_i restarted if a concurrent transaction T_j that is serialized earlier updated data that T_i read.

Algorithm 1 gives the pseudocode of the deterministic execution algorithm that we use, which is based on Aria [15].

Each worker thread picks the next transaction (*current_id*) from the input log and executes the `RUN_Tx` function on the transaction. The function atomically performs two operations: (a) takes the current database snapshot S and (b) records the lowest unfinished transaction-id (*check_from_id* as the last committed transaction ID + 1 (Lines 40 – 41). (We assume that transactions are numbered sequentially in log/commit order.) Executing atomically ensures that all updates from transaction-IDs prior to *check_from_id* are

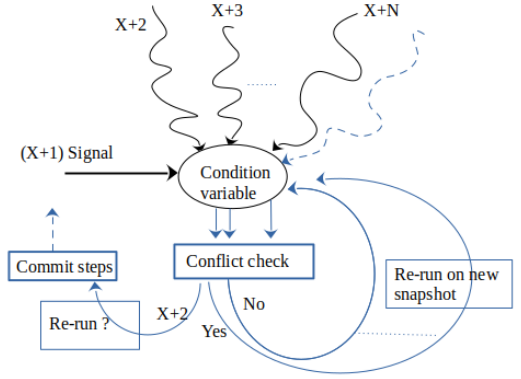


Figure 3: Workers and condition variable

committed in the snapshot. Thus, the current transaction needs to be checked for any possible conflict with transactions that had started but had not finished, before this transaction started.

The rw-conflict check is implemented as described in Aria [15], using code from an implementation of AriaBC on PostgreSQL from github.com/lzllai/AriaBC. As noted in [15], the read-write check also detects phantom conflicts since PostgreSQL uses leaf conflicts for checking conflicts between predicates and updates/deletes/inserts, and these conflicts show up as read-write conflicts on index leaves. We also note that the changes to PostgreSQL above are minimal.

The worker executes the transaction on the database snapshot S , to determine the read-write-set which was not known prior to scheduling the transaction (Line 42). However, it does not commit updates to the database, similar to [14], [15]. A transaction commits updates after verifying that it does not have any data conflicts with the preceding transactions (Line 47).

If there is a conflict the transaction is re-executed by the while loop, after a waiting time as determined in Line 44. The intuition behind the condition variable wait and conflict checking code in the while loop above is depicted pictorially in Figure 3.

If all preceding transactions have committed before snapshot S was taken (the check in the while loop in Line 43), the transaction will commit updates and send a signal to subsequent transactions (higher IDs) that may be waiting (Lines 53, 54). Otherwise, if at least one preceding transaction had not committed before snapshot S was taken, this transaction will wait for the commit signal. This wait can be either for a certain threshold period of time, or a count of commit signals, etc (Line 44). The commit signal indicates the availability of a read-write set from a preceding transaction that committed.

When a commit signal is received, the worker will (a) record the ID of that transaction and (b) compare the read-write set of the current transaction with all those who have published a commit signal (i.e. committed, Line 47). In the optimized case, this translates to the number of signals reaching threshold count or timer expiry. It then compares the read-write sets of all transactions till this recorded ID, with its own read-write set (Line 21).

If there is a conflict, the transaction is restarted by taking a new database snapshot with the latest state. If there is no conflict, and all conflict checks with preceding transactions are done, a transaction

Algorithm 1 Deterministic Transaction Execution Algorithm

```
1:  $maxSetSize \leftarrow K$ 
2:  $RW\_SET[] \text{ setA, setB}$  ▷ global read-write sets
3:
4:  $check\_from\_id \leftarrow 0$  ▷ private to worker
5:  $last\_committed\_tx\_id \leftarrow 0$  ▷ global across threads
6:
7: function COMMIT ( $Tx\_id_i$ )
8:   if ( $mapToSet(Tx\_id_i, maxSetSize) == 0$ ) then
9:     if ( $getSetSize(setA) == maxSetSize$ ) then  $empty(setA)$ 
10:    end if
11:    Insert ( $i, rw\_set_i$ ) into setA
12:  else
13:    if ( $getSetSize(setB) == maxSetSize$ ) then  $empty(setB)$ 
14:    end if
15:    Insert ( $i, rw\_set_i$ ) into setB
16:  end if
17:   $last\_committed\_tx\_id \leftarrow Tx\_id_i$ 
18: return
19: end function
20:
21: function CHECKCONFLICT ( $TX\_ID1, TX\_ID2, RW\_SET_i$ )
22:   $i \leftarrow TX\_ID1$ 
23:  while ( $i \neq TX\_ID2$ ) do
24:    if ( $mapToSet(i, maxSetSize) == 0$ ) then
25:      if ( $(RW\_SET_i \text{ conflicts with } RW\_SET_j \text{ from setA}) \&$   

26:         $(TX\_ID_j \geq check\_from\_id)$ ) then  $return\ true$ 
27:      end if
28:    else
29:      if ( $(RW\_SET_i \text{ conflicts with } RW\_SET_j \text{ from setB}) \&$   

30:         $(TX\_ID_j \geq check\_from\_id)$ ) then  $return\ true$ 
31:      end if
32:    end if
33:     $i \leftarrow i + 1$ 
34:  end while
35: return false
36: end function
37:
38: function RUN_Tx ( $T_k$ ) :
39:   $loop \leftarrow true$ 
40:  while ( $loop$ ) do
41:     $oldest\_commit\_id \leftarrow last\_committed\_tx\_id$ 
42:     $check\_from\_id \leftarrow oldest\_commit\_id + 1$ 
43:     $S \leftarrow DB\_Snapshot()$ 
44:     $rw\_set_k \leftarrow Execute(T_k, S)$ 
45:    while ( $oldest\_commit\_id \neq (k - 1)$ ) do
46:      WaitForCommitSignals()
47:      ▷ periodic or count-based or combination
48:       $latest\_commit\_id \leftarrow last\_committed\_tx\_id$ 
49:       $loop \leftarrow CheckConflict(oldest\_commit\_id,$   

50:         $latest\_commit\_id, rw\_set_k)$ 
51:       $oldest\_commit\_id \leftarrow latest\_commit\_id$ 
52:      if ( $loop$ ) then  $break$ 
53:    end if
54:  end while
55:  end while
56:  Commit( $T_k$ )
57:  CommitSignalBroadcast()
58: end function
```

can commit (Line 7). It commits its changes and saves read-write sets to the global hash table (Line 8 – 16) and then sends a broadcast signal to other workers (Line 54).

The hash table stores read-write sets of all committed transactions. However, once a transaction is old enough that no currently executing transaction is concurrent with it, its read-write sets can be deleted.

To simplify deletion of data from the hash table, we use a pair of hash tables setA and setB, where one table is used till it fills up, and then the other table is used till it fills up. The function mapToSet() (line 8) determines which table to use for the current transaction, based on the transaction ID number. The tables are sized based on the maximum number of threads N , such that when setB becomes full, all current transactions in setA are redundant and can be deleted, and symmetrically for setA becoming full. Due to commit ordering, with N threads the oldest currently executing transaction cannot be more than N transactions before the newest currently executing one.

It is possible that upon re-execution on a new snapshot, the transaction may follow a different codepath from the original simulated execution, resulting in new read-write sets. This is expected to be a rare scenario, and even in this case, the new execution will account for all conflicts.

5 Corruption Detection & Recovery

In this section, we discuss different threat models and the detection of a corrupt replica passively and actively. We present a mechanism to update the Merkle tree with each transaction, along with partitioning. Finally, we describe how to recover a compromised node and discuss integrity constraint violations during recovery.

5.1 Threat Models

There is a range of attacks possible depending on system vulnerability and the attacker's level of sophistication. We consider two threat models reflecting minor and major levels of security breaches.

In the first threat model, we assume the database system code is not compromised. Even when an attacker has broken into the system and tampered with data, the database configuration and automatic update of the Merkle tree remain intact. On the compromised replica, the Merkle tree will generate a different root hash that does not match the state of a healthy replica. When root hashes of all replicas are compared, compromised replicas can be identified as long as they form a minority.

In the second threat model, which we call the major threat model, we consider that the database system code may also be compromised; thus, the Merkle tree hash may not match the underlying data, and the replica may fool a recovering replica into accepting wrong results. Yet, the result of the transaction provided by this replica based on tampered data will not match the majority result, thus indicating a security breach.

To handle this model, we need to first restore the code, then recompute Merkle trees, and then restart the protocol. The rest of our description focuses on the case where code has not been modified.

Activities such as user public key update are done by the system administrator. A compromise of the system administrator's private

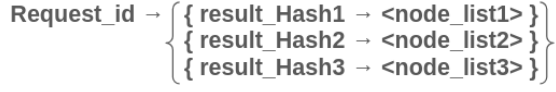


Figure 4: Transaction Result Aggregation Map

key could impact multiple users. We assume in this paper that the administrator credentials are not compromised.

5.2 Corrupt Replica Detection

All replicas receive an identical sequence of globally ordered transactions. In passive detection, the result of each transaction is monitored. If the result from any two database replicas differs, at least one of them must be corrupted. Since only a minority of replicas may be compromised, we can determine the correct transaction result based on the majority. However, data corruption may manifest in transaction result mismatch after a very long time. Hence, a system administrator may want to detect data corruption early, rather than passively waiting for it. We propose a passive and an active scheme below, and later present a mechanism to recover a corrupt replica.

5.2.1 Passive Detection Using Transaction Result. When the result of a transaction execution is found to be different on different replicas, we can infer that one or more replicas have been corrupted.

Each Raft client maintains a data structure shown in Figure 4, which stores for each transaction (request), the set of result hashes received from the replicas, along with the list of replicas that returned that value. This data structure is used to determine the correct value (returned by a majority of the replicas in the system). All replicas whose result hashes differ from the majority are detected as having been corrupted.

This passive detection scheme only detects corruption that affects the result of a query. If some data that has been corrupted is not queried for a long time, that corruption would correspondingly not be detected until it is queried.

5.2.2 Active Detection using Merkle Tree. In the active detection scheme, the database state at all replicas at corresponding points in the input log are compared with each other, by using a Merkle tree whose root node hash represents the state of the entire database. The database state is obtained by taking a database snapshot, and all queries, including queries on Merkle tree data, run on this snapshot. A textbook description of Merkle trees for such comparison may be found in, e.g., [25]

To actively detect a corrupt replica, a special transaction, 'compareStates', is inserted into the blockchain to gather database state at all replicas at the same instant in the transaction log. When a compareStates transaction is found, the application server at the replica does the following atomically: (a) it saves a database snapshot to be used later during the recovery procedure and (b) it publishes the state, represented by the root hash of the Merkle tree (or root hashes of all trees in the partitioned Merkle tree case) of each relation. These states are then compared, and any divergence is used to detect corrupt replicas.

Such active detection can be performed either periodically by the system or initiated on demand by system administrator.

Some database replicas may be unavailable or lagging in execution, and the root hash at that instant may not be available within a reasonable time. Hence, comparison of the rest of the replicas can be done without it, possibly after a timeout. As long as healthy replicas form the majority, we can check such lagging replicas when the root hash at that point in time becomes available.

5.2.3 Merkle Tree Update. Each row in a relation is mapped to a leaf node in the Merkle tree using a hash function. Each time a transaction updates a row, the corresponding Merkle tree leaf value is updated. This leaf update involves removing the contribution of the previous data hash and incorporating the new data hash. This updated value at the leaf node triggers an update to its parent node value, which is done similarly, by removing the contribution of the old leaf hash and incorporating the new leaf hash. This updating of parent node value continues recursively up the tree till the root node. All these updates of Merkle tree nodes are done as part of the original transaction itself, which updated the row.

We note that the Merkle tree nodes are maintained as records of a system relation, ensuring existing conflict detection mechanisms also handle conflicts on the Merkle trees.

Normally, two concurrent transactions updating distinct rows in a user-defined relation can execute independently by locking their respective write-sets.

If the modified rows map to the same Merkle tree leaf, write-sets will imply conflict at all nodes from leaf to root. Even if the modified rows map to different Merkle tree leaf nodes, both transactions will update all parent nodes, up to the Merkle tree root. Thus, write-sets of such transactions will not be disjoint if both impact a common Merkle tree.

Conflicts in updating common items in write-sets, in particular a common root node, would severely limit how many transactions can be processed simultaneously. We therefore use a partitioned Merkle tree.

Partitioned Merkle Tree: To avoid Merkle tree update from being a bottleneck for concurrency, we logically partition each relation and compute a (smaller) Merkle tree on each partition [11]. A transaction updating a Merkle tree leaf now causes updates to its parent nodes only till the root node of its partition. Thus, write-set conflicts between transactions when updating the Merkle tree are significantly reduced.

Figure 5 shows a partitioned tree, with the node ID scheme we use that keeps node IDs unique across all partitions. Node hash data of all nodes across all partitions of the tree is maintained in a single relation, whose rows are identified by the node ID scheme shown.

Transactions modifying user data that map to leaf nodes in the same partition may still conflict while updating Merkle tree nodes, but the partitioning scheme and count can be chosen such that conflicts are greatly reduced. Transactions updating data in different partitions will have no common Merkle tree node, and such updates can be applied independently. On the other hand, a transaction updating data across multiple partitions will limit the concurrency gain offered by partitioning. In all cases, the transactions commit in the same global input log order defined by the blockchain.

When the content of a database replica is compromised, as identified by a difference from other replicas in either Merkle tree root

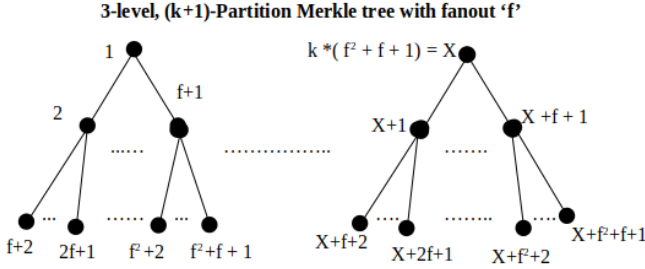


Figure 5: 3-level, (k+1) Partitioned Merkle tree with node-IDs

Algorithm 2 Recovery Algorithm

```

1: procedure RECOVERY (REPLICA  $N_k$ )
2:                                      $\triangleright$  active detection start
3:   mark_log_offset  $\leftarrow$  current_log_offset ('compareStates' transaction)
4:   For each replica  $N_i$ :
5:      $S_i \leftarrow DB\_Snapshot(replica\ N_i, mark\_log\_offset)$ 
6:      $M_i \leftarrow Merkle\_Tree(S_i)$ 
7:   [ $G_{majority}, G_{rest}$ ]  $\leftarrow Group\_replicas\_by\_merkle\_root\_hash()$ 
8:                                      $\triangleright$  active detection completed
9:                                      $\triangleright$  recovery phase start
10:  ( $N_{ref}, M_{ref}$ )  $\leftarrow$  reference instance of ( $replica\_ids, G_{majority}$ )
11:  Compare_merkle_trees_and_Replicate_rows( $M_k, M_{ref}, N_k, N_{ref}$ )
12:  Replay_input_tx_logs_and_rejoin_cluster(mark_log_offset)
13:                                      $\triangleright$  recovery phase complete
14: end procedure

```

hash or a transaction result, that indicates the need for a recovery mechanism to bring the replica back to operational state.

5.3 Corrupt Replica Recovery

The state of a database replica may be corrupted by malicious updates. System failures, like power or network, or component failure, which result in a replica falling substantially behind the rest of the system can also be treated similar to corruption, to recover the state, without playing back a very long transaction log.

Before initiating recovery, we first stop processing at the corrupted replica and ensure that the intruder no longer has access to the system. With manual intervention, the security loophole on the corrupted replica is fixed, like a security patch or system update, to prevent it from being compromised again in the future.

We describe the procedure performed by the system administrator to recover a corrupted replica pictorially in Figure 6, and with pseudocode in Algorithm 2.

The first step is to identify a healthy replica to use as a reference. To do so, a logical snapshot S_1 of the current database state is taken at all replicas (Line 4,5), as of a certain point in the transaction log as explained in Section 5.2.2. The logical snapshot S_1 includes the user data as well as corresponding Merkle trees. The default mechanism for taking snapshots in PostgreSQL takes care of this step.

We choose one of the majority of replicas which have an identical Merkle tree root, thus an identical database state, as the reference replica (Line 10).

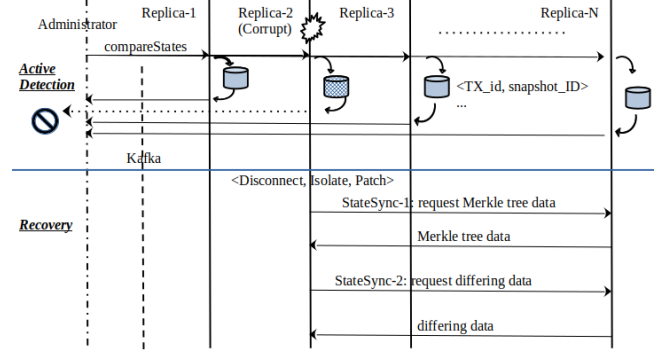


Figure 6: Procedure to recover a failed replica

Next, we identify the differences between the Merkle trees of the corrupted replica and the reference replica (Line 11) on the snapshot S_1 taken in the earlier step. Since the root hash of the corrupted replica is different, the hash of at least one of the children of the root node must differ from that in the reference replica. By recursively comparing the hashes of children down the tree, we can identify leaf-nodes whose hashes do not match across replicas. In case of partitions, as proposed in the previous section, the tree comparison must be done between all corresponding partition-trees.

From mismatching leaf-nodes, we identify differing rows in the database by using an index mapping Merkle tree leaf nodes to the rows whose hash values map to the node. Such an index is easily implemented in PostgreSQL using a “function index” on the table, wherein an index is created with its key as a function that maps the row to the Merkle tree leaf identifier. The differing rows are then resolved by replicating the state of those rows from snapshot S_1 of a reference database replica.

At the end of the above phase the partition has been recovered up to the point in input transaction log when the snapshot was taken. The state of the erstwhile corrupted replica is then brought to the current state by replaying the input transaction log starting from just after the point in the log where the snapshot was taken, during the active detection procedure (Line 12).

It is possible to restore a minority of corrupted replicas concurrently with this procedure, even if they were corrupted at different times. Also, a healthy replica may serve as a reference for multiple corrupt replicas simultaneously.

Our mechanism utilizes snapshots, which most current generation databases support. In PostgreSQL, we can get a specific snapshot with an identifier, and then run multiple queries on that snapshot. We use this feature in our implementation.

5.3.1 Integrity Constraint Discussion. Note that the database state at the healthy replica was reached by executing transactions in the order they appeared on the blockchain. However, the recovery program imports all the differences from the healthy reference replica to the corrupt replica, without any order. Hence, while the recovery program is being run, the corrupt database instance may temporarily violate integrity constraints. For example, a relation P may refer to a foreign key in relation Q . A transaction may add new entries to both relations on a healthy replica. However, the

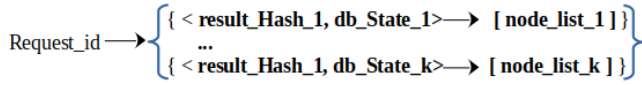


Figure 7: Transaction Result and State Aggregation Map

recovery program may first update relation P entirely, while the foreign key entry in relation Q is yet to be updated.

Hence, constraint checks are disabled while the recovery is in progress and re-enabled when complete. After all differences are applied, the recovered replicas will satisfy all constraints, and integrity checks are re-enabled.

6 Handling Non-Deterministic Transactions

The architecture of our system assumes that transactions are deterministic. However, a badly written transaction may be non-deterministic; for example, the execution result may depend on a random number, or dynamic system-parameters like time, thread-id, etc. In this case, despite executing transactions in the same deterministic order across all replicas, database states across replicas could diverge. This is different from database corruption due to a security breach, which was discussed earlier, and could result in each replica returning a different result, or having a different state after the transaction.

Our system already compares the query results from different replicas, and will detect the case where the query result is non-deterministic. However, transactions may update the database non-deterministically, resulting in divergent states across replicas, while returning a deterministic query result.

We present below detection and recovery mechanisms for such non-deterministic transactions.

6.1 Detecting Nondeterminism

To detect database state mismatch across replicas due to a non-deterministic transaction, we propose a simple extension of the scheme used earlier for passive detection of data corruption. Writes to the database only return an execution status, and even if the actual writes are different in different replicas, the status may be identical.

To detect divergence, in addition to returning transaction results, each replica now also sends the updated Merkle tree root hash of all the partitions that are updated, for each updated relation. The partition root hashes are compared, along with result hashes, to detect divergence.

Figure 7 shows an extension of the Figure 4 for detecting database state divergence. The client collects the result hash and updated partition hash information (i.e. partition ID and root hash) for all partitions updated by the transaction.

We would like to highlight that this mismatch is detected immediately after the commit of a nondeterministic transaction. Note that this extended scheme can also be used to improve accuracy of passive corruption detection described earlier, in Section 5.2.1.

The above detection mechanism is likely to be particularly useful for debugging transactions, since it catches non-determinism immediately, allowing easy detection of the cause for non-determinism.

In a well designed system there should ideally be no occurrence of non-determinism, but the check could be done anyway since it is very low overhead, similar to `assert()` function calls in programs.

However, in case non-deterministic execution does happen in a production system it is important to be able to recover quickly from it. We describe how to do so next.

6.2 Recovery process

When a database state divergence is detected across replicas, it could be either because of a security breach or non-determinism. While the underlying cause for the non-deterministic execution of the transaction needs to be fixed, if it was a rare error, example due to a “heisenbug”, there may not be another occurrence for a while, giving time to fix the bug. A repeated error would, however, cause slowdown of the system due to recovery, and need urgent fixing.

As long as a majority of the replicas give the same result, the recovery method described in Section 5.3 can be used without any change. However, it is possible that there is no majority group. In that case, recovery can be initiated after deciding which replica to use as the correct copy, which can be done by either choosing one which is in a majority, or if there is no majority, by a simple majority, or by decision of a system administrator.

The recovery mechanism proposed earlier for corruption can be applied here with minor enhancements as described below. The key differences in the recovery mechanism are (a) how reference replica is chosen based on Merkle root hash (or in case of partitioned tree, partition root hash) values across replicas and (b) recovery may be done for multiple replicas (in the limiting case, for all replicas but one).

In case some replicas report common states, we could use the set of replicas with highest cardinality (if one exists) as reference. As observed earlier, in the extreme case, each replica may give a different root hash. In this case, there is no single source of truth and we can choose any one as reference. Those replicas whose database states are different from reference replicas, are referred hereafter as diverged replicas.

A special transaction ‘saveSyncState’ is inserted into the shared input log, listing all reference replicas. It causes each replica to save database snapshot and check if it is listed as reference replica. If a replica is not a reference, it needs to stop processing and recover state from one of the reference replica listed, and after that it can fetch newer requests from the shared input log. The replica listed as reference can simply save the snapshot, and continue to process newer requests without interruption.

Note that non-determinism is detected after transactions have committed at the replicas. Transactions added to the shared input log before the special recovery transaction is inserted will be processed by the replicas, and some of these may have a dependency on the non-deterministic transaction. Since our detection mechanism is quick and no manual intervention is necessary, there will be very few such transactions. Database state changes due to these transactions, as well as those due to the non-deterministic transaction, are replicated from a reference replica and would be consistent after recovery. But results returned to users based on these transactions could potentially have errors.

If a transaction does not depend on the non-deterministic update, the majority would have returned the same result as the reference value. If it did, and there is a majority with the same value for the non-deterministic result, the dependent transaction would have returned the same result on all these replicas. If there is no majority, the dependent results would also not be in a majority except in an unlikely scenario where divergent inputs gave the same transaction result for the dependent transaction. In this unlikely case alone we may need to inform a user that the value returned may be changed.

7 Implementation Details

In this section, we describe several implementation details.

7.1 Transaction flow through System

Transaction requests are sent by user clients to Ratis clients; Ratis [10] is an open source implementation of Raft. Requests arriving at the Raft client are sent to a Raft server, which uses consensus with other server instances to determine a shared order, in a crash fault tolerant manner. In our implementation, each Raft server replica has a unique ID and is configured and initialized identically with a Raft server instance, an application server, and a PostgreSQL database. The Raft server instance on each node sends transactions that have reached commit status on the Raft log to the application server. The server is multi-threaded, allowing concurrent execution of multiple transactions (deterministic ordering is handled at the database side). Database requests are sent to the local PostgreSQL database server through the connection pool. The number of threads and size of the pool is configured to be the same as the number of workers on the PostgreSQL server.

The application server receives query results from the PostgreSQL server, and on completing the transaction, publishes the transaction result to Apache Kafka result topic [9] along with its server ID. During corruption recovery, Administrator connects to application server to (a) initiate Merkle tree comparison (b) lookup data items using functional index and (c) replay transaction log.

Kafka service has separate topics to hold transaction results, and server errors, respectively. The Ratis client which received a user transaction gathers results for the transaction from all replica servers via the Kafka result topic and stores them in a hash-map as described earlier. If the client detects a divergence in results from one or more replicas it publishes a message to a Kafka error topic to trigger recovery actions by the Administrator.

7.2 Database processing

Database executes transactions deterministically as per our algorithm, which is based on the implementation of Aria [15] in Postgres, provided by the authors of HarmonyBC [14].

7.2.1 Initialization. We create schema-specific user relations, functions on each Postgres database replica, with all constraints as deferrable. For each user relation, we create a Merkle tree, implemented as an index. All relations and Merkle trees are set to the same initial state across replicas.

Rows in a user relation are mapped uniformly with a hash function to Merkle relation leaf nodes. The hash of each user data row is computed by hashing attribute values with Blake3. Hashes of all rows mapping to a leaf node are combined using the XOR function

to allow efficient incremental update of the leaf hash values. Hash-value of a parent node is computed by combining the hash values of all child nodes again using the XOR function. The Merkle tree is configured identically on all replicas (schema, number of leaf nodes, fanout, mapping and hashing functions, partitions, etc).

7.2.2 Merkle Tree. We created a new index type in PostgreSQL to maintain the Merkle trees. For each update done to the user relation by a transaction, the index update is invoked by the database code, and the index implementation updates the corresponding rows of the Merkle tree relation. Normal concurrency control and recovery techniques get applied on the relation representing the Merkle tree.

For partitioned Merkle trees, one relation holds the hash values of Merkle tree nodes of all partitions, each with a different ID range. With P partitions, fanout of F and k levels in the Merkle tree, there will be P root nodes, and node count $N = (F^{k+1} - 1) / (F - 1)$ per partition and $P * F^k$ total leaf nodes and Merkle relation with $P * N$ rows. For example, Merkle tree partition with 4 levels and a fanout of 8, has 512 leaf nodes and 585 total nodes in the partition. We vary the number of partitions from 100 to 1000 in order to bring down the data conflicts between transactions by orders of magnitude.

7.3 Corruption recovery

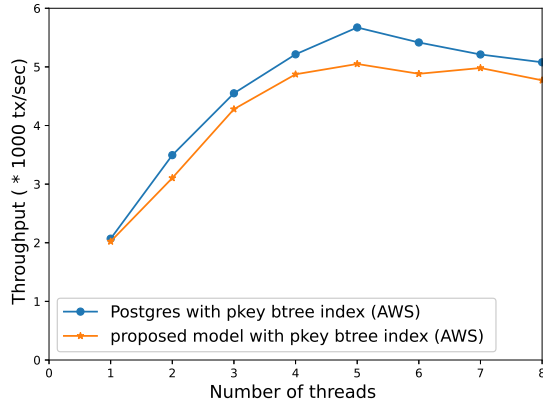
The system administrator inserts a special 'compareStates' transaction into the input log, to compare the database state at all replicas at the same instant in the transaction log, say P_{tx} . The transaction atomically (a) saves a logical database snapshot S_t when all preceding transactions have committed and (b) publishes as result, the root hash of the Merkle tree (or root hashes of all trees in the partitioned case). The snapshot S_t includes all database relations, including the user relation and the corresponding Merkle trees.

As mentioned earlier, one of the healthy replicas is chosen as the reference replica N_{ref} . The recovery program simultaneously connects to replica N_{ref} with a read-only session and to the corrupt replica with a read-write session. On the corrupt replica, commands are sent to (a) disable new connections, (b) disconnect all connections other than the recovery session, and (c) disable / defer the constraints. On both replicas, (a) the database snapshot is set to S_t and (b) Merkle node hashes are requested (of all partitions in partitioned case). Merkle trees at the corrupted replica and the replica N_{ref} are recursively compared down from the root node, to find all leaf nodes whose hashes differ.

From mismatching leaf-nodes, we identify differing rows in the database by reverse-lookup using functional index on snapshot S_t . The differing rows are then replicated from snapshot S_t of replica N_{ref} . The state of the erstwhile corrupted replica is then brought to the current state by replaying the input transaction log starting from the point P_{tx} where the snapshot S_t was taken. This is very efficient except in case the corruption is major, when it is best to copy over a snapshot without Merkle tree comparison.

8 Performance Evaluation

We compare the performance of our proposed model on local and cloud setups, on the popular OLTP benchmark, YCSB. The local node in our setup is an AMD Ryzen7 5700G with 16 cores, NVMe SSD, and 16GB RAM running 64-bit Ubuntu 22.04 LTS. On AWS, results are obtained using *m5d.4xlarge* EC2 instances, with Ubuntu



Performance with signed queries, Merkle trees and passive detection: with and without determinism

Figure 8: Impact of Determinism

24.04 and SSD storage with Linux kernel version 6.14.0-1018-aws and GCC version 13.3.0

Traffic generator Benchmark [7] generates transactions in a pre-configured proportion of operations. It supports multiple benchmarks like YCSB, Smallbank, and TPC-C, each of which defines its own schema.

RSA key-pairs are generated with a common configuration (PKCSv15 padding, 2048 key-length, UTF-8 format, Base64 encoding, etc) for both client and system, and public keys are published to the destination.

8.1 Evaluation on single node

We first conduct an ablation study of different factors on a single machine.

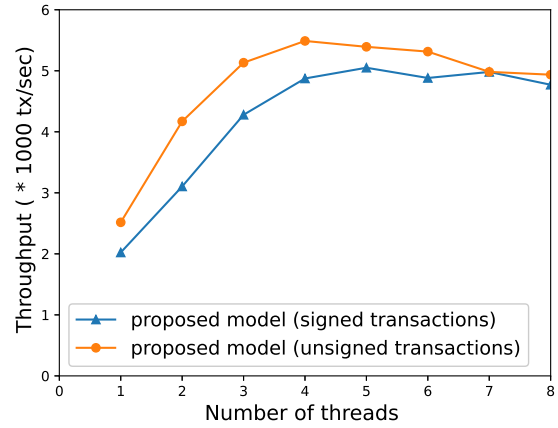
8.1.1 Impact of determinism. We compare our proposed deterministic model with the baseline of (non-deterministic) Postgres with Serializable transaction isolation, on a local machine with SSD storage.

As seen in Figure 8, the throughput initially increases linearly with number of threads. For higher number of threads (greater than 5), memory IO becomes a limiting factor, and contention between threads leads to a slight dip in throughput.

Our proposed deterministic database model gives a peak performance of around 5000 transactions per second, which is slightly lower than that of native PostgreSQL; performance is within 10% of baseline PostgreSQL across the number of concurrent threads considered. The performance closely matches that of other deterministic database implementations, as reported in Harmony[14].

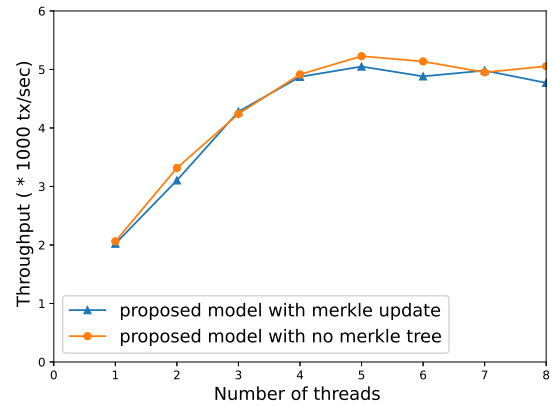
8.1.2 Impact of signed transactions. As seen in Figure 9, adding signatures reduces throughput a little with a smaller number of threads, but as the number of threads increases, this difference vanishes. Since signature creation is CPU-bound it benefits from extra threads.

8.1.3 Impact of maintaining Merkle tree. For each user relation (e.g. usertable in YCSB), a Merkle tree is constructed with 200 partitions



Performance with determinism, Merkle trees and passive detection: with and without signing of transactions

Figure 9: Impact of RSA-signing of transactions



Performance with determinism, passive detection, and signing of transactions: with and without Merkle trees.

Figure 10: Impact of maintaining Merkle tree

and 16 leaf nodes per partition. As seen in Figure 10, throughput increases linearly and then saturates in both cases, with and without Merkle tree. However, there is not much difference in system throughput in the two cases. This is probably because the Merkle tree nodes are frequently accessed and would be in buffer, so the IO overhead is minimal.

8.2 End-to-end system performance

We evaluate our proposed model on a multi-node setup. We measure the impact of adding Kafka alone, with transactions already serialized (as is the case in some applications), and the impact of adding Raft consensus for determining the serialization order along with Kafka. Our Raft implementation needs Kafka to collect the execution results, so we do consider the case of Raft alone.

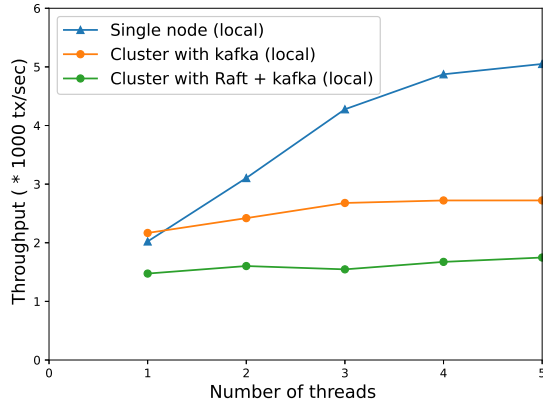


Figure 11: Performance of End-to-end system

Raft consensus algorithm, as implemented by Apache Ratis is used across 4 replicas. One of the replica nodes is configured with Kafka broker. A transaction is considered complete when its results are received by the Raft client from a threshold number of servers (in our case 3), and they match.

Figure 11 shows the throughput numbers for the single node case, compared with cluster with Kafka alone, and cluster with Raft and Kafka, with different numbers of threads at the client. We can see that on a cluster the performance with Kafka alone as well as with Raft and Kafka do not increase with number of threads, indicating that the system is not CPU bound. Both Kafka and Raft require multiple rounds of messaging, which appears to be the bottleneck. Further, using Raft reduces the throughput compared to using Kafka without consensus, indicating that Raft is currently the bottleneck.

8.3 Corruption Recovery performance

We evaluate performance of proposed corruption recovery mechanism using YCSB benchmark with 1 million tuples with 200 Merkle tree partitions. With 16 leaf nodes per partition, nearly 300 tuples map to a Merkle leaf node. To simulate corruption of k leaf nodes, 300 corrupted tuples are distributed uniformly among the k leaf nodes. As seen in Figure 12, the recovery time is only a few seconds, and increases linearly with number of leaf nodes modified. The recovery time is much less than the time for copying the entire data (which could take hours or days on very large data) as is done by some earlier systems, since it only copies corrupted tuples.

8.4 Discussion

We observe that signature verification, Merkle tree maintenance and support for determinism impose relatively small overheads on transaction processing throughput. On the other hand, both Raft consensus and Kafka add significant overheads on system throughput. Optimizing these components to improve throughput is an area of ongoing work.

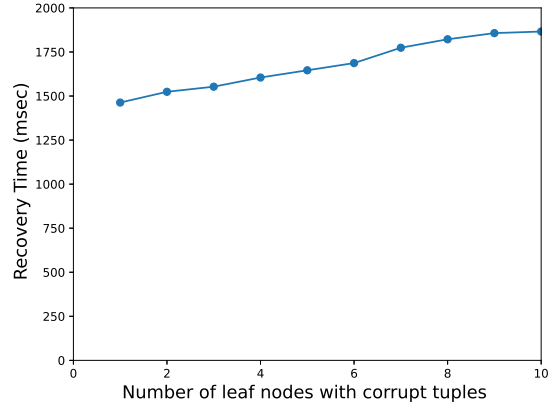


Figure 12: Performance of Recovery from Corruption

9 Conclusion

We presented a design for protecting database data from corruption, based on the Byzantine-fault tolerant replicated state machine approach. As long as only a minority of replicas are compromised, our design supports recovery of the corrupt replicas while the system is fully operational, without any downtime. We show that the approach can be implemented quite efficiently, on top of PostgreSQL, with relatively small changes to PostgreSQL itself. The system provides throughput of over 2500 YCSB transactions per second, and recovery is shown to be very efficient.

We believe that our approach will have significant practical impact, given the increased risks of attacks such as ransomware or advanced penetration attacks that compromise servers.

Acknowledgments

Authors would like to thank Neel Parekh for contributing the index implementation of Merkle tree in PostgreSQL.

References

- [1] D. Abadi and J. Faleiro. *An overview of deterministic database systems*. Communications of the ACM, vol.61, 2018.
- [2] Daniel J. Abadi and Jose M. Faleiro. *An overview of deterministic database systems*. *Commun. ACM*, 61(9):78–88, 2018.
- [3] Elli Androulaki, Artem Barger, Vita Bortnikov, Christian Cachin, Konstantinos Christidis, Angelo De Caro, David Enyeart, Christopher Ferris, Gennady Laventman, Yacov Manevich, Srinivasan Muralidharan, Chet Murthy, Binh Nguyen, Manish Sethi, Gari Singh, Keith Smith, Alessandro Sorniotti, Chrysoula Sthakopoulou, Marko Vukolić, Sharon Weed Cocco, and Jason Yellick. *Hyperledger fabric: a distributed operating system for permissioned blockchains*. In *Procs. EuroSys Conf.*, 2018.
- [4] P. Antonopoulos, R. Kaushik, H. Kodavalla, S. Rosales Aceves, R. Wong, J. Anderson, and J. Szymaszek. *SQL Ledger: Cryptographically Verifiable Data in Azure SQL Database*. In *Proceedings of the ICDM, ACM SIGMOD*, 2021.
- [5] M. Bellare and B. Yee. *Forward integrity for secure audit logs*. Technical report, UC San Diego, 1998.
- [6] P. Devanbu, M. Gertz, C. Martel, and S. Stubblebine. *Authentic Data Publication over the Internet*. *Journal of Computer Security*, 11, 2002.
- [7] D. Difallah, A. Pavlo, C. Curino, and P. Cudre-Mauroux. *Oltp-bench: An extensible testbed for benchmarking relational databases*. *Proceedings of the VLDB*, 2013.
- [8] T. Distler. *Byzantine fault-tolerant state-machine replication from a systems perspective*. *ACM Comput. Surv.*, 54(1), February 2021.
- [9] Apache Foundation. *Apache Kafka*, <https://kafka.apache.org/>, 2025.
- [10] Apache Software Foundation. *Apache Ratis*. <https://ratis.apache.org>, 2025.
- [11] A. Freij, H. Zhou, and Y. Solihin. *Bonsai Merkle Forests: Efficiently Achieving Crash Consistency in Secure Persistent Memory*. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021.

- [12] Hector Garcia-Molina, Frank M. Pittelli, and Susan B. Davidson. Applications of Byzantine agreement in database systems. *ACM Trans. Database Syst.*, 11(1):27–47, 1986.
- [13] Ilir Gashi, Peter T. Popov, Vladimir Stankovic, and Lorenzo Strigini. On designing dependable services with diverse off-the-shelf SQL servers. In *WADS, volume 3069 of Lecture Notes in Computer Science*, Springer, pages 191–214. 2003.
- [14] Z. Lai, C. Liu, and E. Lo. When private blockchain meets deterministic database. *Proceedings of the ACM on Management of Data*, 2023.
- [15] Y. Lu, X. Yu, L. Cao, and S. Madden. Aria: a fast and practical deterministic oltp database. *Proceedings of the VLDB*, 13, 2020.
- [16] Aldelir Fernando Luiz, Lau Cheuk Lung, and Miguel Correia. MITRA: byzantine fault-tolerant middleware for transaction processing on replicated databases. *SIGMOD Rec.*, 43(1):32–38, May 2014.
- [17] E. Mykletun, M. Narasimha, and G. Tsudik. Authentication and integrity in outsourced databases. *ACM Trans. Storage*, 2(2):107–138, May 2006.
- [18] Senthil Nathan, Chander Govindarajan, Adarsh Saraf, Manish Sethi, and Praveen Jayachandran. Blockchain meets database: Design and implementation of a blockchain relational database. *Proc. VLDB Endow.*, 12(11):1539–1552, 2019.
- [19] D. Ongaro and J. Ousterhout. In *Search of an Understandable Consensus Algorithm*. In *Annual Technical Conference 2014*. USENIX Association, 2014.
- [20] Ricardo Padilha, Enrique Fynn, Robert Soule, and Fernando Pedone. Callinicos: Robust transactional storage for distributed data structures. In *USENIX Annual Technical Conf.*, 2016.
- [21] Ricardo Padilha and Fernando Pedone. Augustus: scalable and robust storage for cloud applications. In *Proceedings of the 8th ACM European Conference on Computer Systems*, EuroSys '13, page 99–112, New York, NY, USA, 2013. Association for Computing Machinery.
- [22] Nuno Preguic and Rodrigo Rodrigues amd Cristovao Honorato amd Joao Lourenco. Byzantium: Byzantine-Fault-Tolerant Database Replication Providing Snapshot Isolation. In *Procs Workshop on Hot Topics in System Dependability (HotDep)*, Usenix, 2008.
- [23] Kun Ren, Alexander Thomson, and Daniel J. Abadi. An evaluation of the advantages and disadvantages of deterministic database systems. *Proc. VLDB Endow.*, 7(10):821–832, 2014.
- [24] Felix Schuhknecht, Ankur Sharma, Jens Dittrich, and Divya Agrawal. chainifyDB: How to get rid of your blockchain and use your dbms instead. In *CIDR*, 2021.
- [25] A. Silberschatz, H. F. Korth, and S. Sudarshan. *Database system concepts*. McGraw-Hill, 7 edition, 2022.
- [26] Florian Suri-Payer, Matthew Burke, Zheng Wang, Yunhao Zhang, Lorenzo Alvisi, and Natacha Crooks. Basil: Breaking up bft with acid (transactions). In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, SOSP '21, New York, NY, USA, 2021. Association for Computing Machinery.
- [27] Florian Suri-Payer, Neil Girdharan, Liam Arzola, Shir Cohen, Lorenzo Alvisi, and Natacha Crooks. Pesto: Cooking up high performance bft queries. In *Procs. ACM SIGOPS Symp. on Operating Systems Principles*, SOSP '25, page 529–554, 2025.
- [28] Alexander Thomson and Daniel J. Abadi. The case for determinism in database systems. *Proc. VLDB Endow.*, 3(1):70–80, 2010.
- [29] Ben Vandiver, Hari Balakrishnan, Barbara Liskov, and Samuel Madden. Tolerating byzantine faults in transaction processing systems using commit barrier scheduling. In *Procs. ACM Symposium on Operating Systems Principles SOSP*, pages 59–72, 2007.
- [30] X. Yang, Y. Zhang, S. Wang, B. Yu, F. Li, Y. Li, and W. Yan. Ledgerdb: A centralized ledger database for universal audit and verification. *Proc. VLDB Endow.*, 13, 2020.
- [31] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham. Hotstuff: Bft consensus with linearity and responsiveness. In *In Proc. ACM PODC*, 2019.
- [32] C. Yue, M. Zhang, C. Zhu, G. Chen, D. Lohin, and Beng C. Ooi. Veribench: Analyzing the performance of database systems with verifiability. *Proc. VLDB Endow.*, 16, 2023.
- [33] Irene Zhang, Naveen Kr. Sharma, Adriana Szekeres, Arvind Krishnamurthy, and Dan R. K. Ports. Building consistent transactions with inconsistent replication. In *Proceedings of the 25th Symposium on Operating Systems Principles*, SOSP '15, page 263–278, New York, NY, USA, 2015. Association for Computing Machinery.
- [34] Liangrong Zhao, Hans Schmiedel, Qin Wang, and Jiangshan Yu. Janus: Enhancing asynchronous common subset with trusted hardware. In *2024 Annual Computer Security Applications Conference (ACSAC)*, pages 488–504, 2024.